

一、名词解释 第一章

1. 操作系统

是指控制和管理整个计算机硬件和软件资源,并合理地组织调度计算机的工作和资源的分配,以提供给用户和其他软件方便的接口和环境,是计算机系统中最基本的系统软件。

2. 实时操作系统

系统能及时响应外部事件的请求,在规定的时间内完成对该事件的处理,并控制所有实时任务协调一致地运行

3. 互斥共享:

多个进程或线程可以共享一个资源,但同一时间只有一个进程或线程能够访问该资源。其他进程或线程必须等待,直到当前访问的进程或线程释放资源才能获得访问权限

三、填空

1. 硬件系统和软件系统 2. 临界资源 共享资源 3. 共享性, 异步性

第二章

- 一、3. 临界区: 对公共资源进行访问的程序片段

4. 进程同步: 有进程同步的目标是协调多个相关进程的执行顺序以确保它们能够按照一定的规则共享系统资源并有效合作

- 三. ④ 1. 动态性 ~~独立~~ 异步性 结构性 5. 动态 - 静态

6. 资源竞争

7. 共享存储, 消息传递, 管道通信 8. 就绪, 运行, 阻塞

- 五. 2. $S_1 \rightarrow S_2$ a_1 $S_4 \rightarrow S_7$ d Semaphore CoBegin
 $S_1 \rightarrow S_3$ a_2 $S_5 \rightarrow S_7$ e S_1 { 完成动作 S_1
 $S_2 \rightarrow S_6$ b_1 $S_6 \rightarrow S_7$ f $V(S_1, S_2) V(a_1)$
 $S_2 \rightarrow S_5$ b_2 $\{ V(S_1, S_5) V(a_2)$
 $S_3 \rightarrow S_6$ c S_2 { $\overline{a_1}$
 $\overline{P(S_1, S_2)} P(a_1)$
 $\overline{P(S_1, S_5)}$ 完成动作 S_2
 $\overline{V(b_1)} V(b_2)$


```

S3 { 完成动作 S3 P(a);
    V(c);
}
S4 { 完成动作 S4 P(b);
    V(d);
}
S5 { 完成动作 S5 P(b);
    V(e);
}
S6 { P(c);
    完成动作 S6 V(f);
}

```

Coend

3. Semaphore empty = 5, orange = 0, apple = 0, mutex = 0

```

Dad() { while(1) { wait(empty);
    wait(mutex);
    将水果放入盘子; signal(mutex);
    if(放入桔子) signal(orange);
    else signal(apple);
}
}

```

```

Son() { while(1) { wait(orange);
    wait(mutex);
    从盘中取一个桔子; signal(mutex);
    signal(empty);
    享用桔子;
}
}

```

```

Daughter() { while(1) { wait(apple);
    wait(mutex);
    从盘中取一个苹果; signal(mutex);
    signal(empty);
    享用苹果;
}
}

```

CoBegin

PA; PB; PC;

Coend

4. avail 1, avail 2 初始值
full 1, full 2 初始值 0

PA:

begin

L1: read from disk;

P(avail 1);

put to buffer 1;

V(full 1);

goto L1;

End;

PB:

begin

L2: P(full 1);

get from buffer 1;

V(avail 1);

P(avail 2);

put to buffer 2;

V(full 2);

goto L2;

~~End~~ End;

PC:

begin

L3: P(full 2);

get from buffer 2;

V(avail 2);

print RECORD;

goto L3 end;

6. 信号量 $customer = 0$;
 信号量 $server = 0$;
 信号量 $mutex = 1$;

```
int waiting = 0;
Server() { while(1) {
    P(customer);
    P(mutex);
    叫号;
    waiting--;
    V(mutex);
    V(server);
    提供服务;
}
```

```
Customer() { P(mutex);
    if (waiting < n) {
        取号;
        waiting++;
        V(mutex);
        V(customer);
        被服务;
    }
    else { V(mutex);
        离店;
    }
}
```

第三章 2. 当有一堆任务要处理但由于资源有限这些事无法处理。这就需要确定某种规则来决定处理这些任务的顺序，这就是处理机调度

3. 是指作业被提交给系统开始到作业完成为止这段时间间隔

4. 各进程互相等待对方手里的资源导致各进程阻塞无法向前推进

三. 2 后备运行和完成

6. 预防. 避免. 检测. 解除

五. 1. PA: 3. PB: 7 PC: 9

PD: 12 PE: 12.

完成 周转

FCFS 3. 9. 13. 18. 20 3. 7. 9. 12. 12

STF 3. 6. 14. 20. 25 3. 2. 10. 18. 19

带权 平均

FCFS 1. 7/6 9/6 8. 6

12/5 6

STF 1. 1. 2. 5. 3. 3. 8 3. 8. 10. 8

平均带权

FCFS $\frac{1+1.17+2.25+2.6+6}{5} = 2.706$

STF $\frac{1+1+2+3+3.8}{5} = 2.26$

五. 1. (1) FCFS: A → B → C → D → E

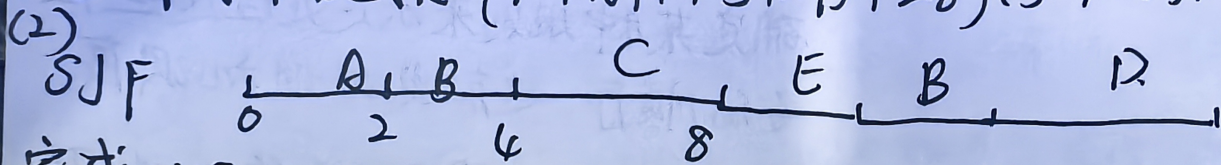
① 完成时间: A: 3. B: 3+6=9 C: 9+4=13 D: 13+5=18 E: 18+2=20

② 周转时间 A: 3-0=3 B: 9-2=7 C: 9+4=13 D: 13+5=18 E: 20-8=12

③ 带权周转 A: 3/3=1 B: 7/6=1.167 C: 9/6=1.5 D: 12/5=2.4 E: 12/2=6

④ ~~平均带权~~ 周转 $(3+7+3+11+10) \div 5 = 7.6$

⑤ 平均带权周转 $(1+1.167+1.5+2.75+2.8) \div 5 = 1.8434$



完成时间: $A: 3$ $B: 15$ $C: 8$ $E: 10$ $D: 12$

周转时间: $A: 3-0=3$ $B: 13$ $C: 4$ $E: 2$ $D: 14$

带权周转 $A: 1$ $B: 2.167$ $C: 1$ $E: 1$ $D: 2.8$

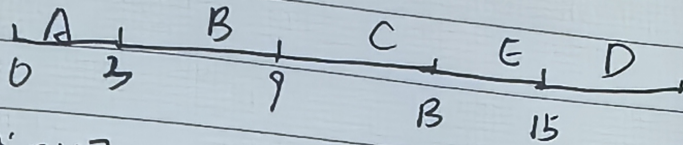
平均周转: ~~$A: 3+3+3$~~ $(3+13+4+2+14) \div 5 = 7.2$

平均带权周转 $(1+2.167+1+1+2.8) \div 5 = 1.5934$

(3) HRRN

Date.

NO.



完成时间: A:3 B:9 C:13 E:15 D:20

周转时间: A:3 B:7 C:9 E:7 D:14

带权周转时间 $(3+7+9+7+14) \div 5 = 8$

平均带权周转时间 $(1+1.167+2.25+2.8+3.5) \div 5 = 2.1434$

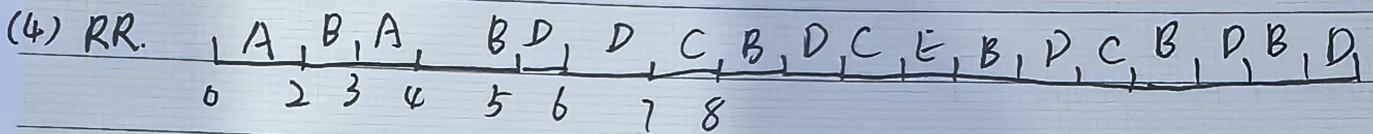
响应比: $B = \frac{1+6}{6} = \frac{7}{6}$

$C = \frac{9}{9}$

$D = 1.6$

$E = 1.5$

$C > D > E$



完成时间: A:4 B:19 C:16 D:20 E:13

周转时间: A:4 B:17 C:12 D:14 E:5

带权一: A:1.33 B:2.83 C:3 D:2.8 E:2.5

平均周转: $(4+17+12+14+5) \div 5 = 10.4$

平均带权周转: $(1.33+2.83+3+2.8+2.5) \div 5 = 2.602$

2. A松弛度: $20-10=10ms$

B: $50-10=40ms$

C: $50-15=35ms$

A的松弛度最低. 执行A. 后: B: $40-10=30ms$ C: $35-10=25ms$

C的松弛度最低. 执行C后. 执行B

A $\rightarrow$ C $\rightarrow$ B

3. (1) Allocation = MAX-need

process	Allocation			
P ₀	0	0	3	2
P ₁	1	0	0	0
P ₂	1	3	5	0
P ₃	0	3	3	2
P ₄	0	0	1	0

系统安全

(2) ① $Request_2(1, 2, 2, 2) \leq Need_2(2, 3, 5, 6)$

② $Request_2(1, 2, 2, 2) \leq Available(1, 6, 2, 2)$

③ $Available = (0, 4, 0, 0)$

$Allocation(2, 5, 7, 6)$

$Need_2(1, 1, 3, 4)$

④ 进行安全性检查：此时对所有进程，条件 $Need_i \leq Available(0, 4, 0, 0)$

不成立，不满足安全状态，因此当 P₂ 提出请求 Request(1, 2, 2, 2) 后，系统不能将资源分配给它。

(3) 系统立即满足 P₂ 的请求 (1, 2, 2, 2) 并未马上进入死锁

上述过程未申请到新资源，并且得不到资源进入阻塞态